CS336 Assignment 5: Reasoning RL

Writeup

Lenard Strahringer

May 30, 2026

Contents

1 Prompting

Problem (prompting_baselines)

Run OLMo-2-0425-1B on GSM8K (5 points).

(a) Evaluation script and metrics

The script `scripts/eval_prompting_baselines.py` starts a vLLM server (`allenai/OLMo-2-0425-1B`), generates one completion per problem on the full GSM8K test split (1,319 problems) using temperature 1.0 and a 512-token budget, and scores each generation with the appropriate reward function from `cs336_alignment.drgrpo_grader`: `question_only_reward_fn` for `question_only` (expects a final answer in `\boxed{}`) and `r1_zero_reward_fn` for the two `r1_zero` variants (expects `</think> <answer> ... </answer>` format). Every generation is bucketed into one of three categories: Cat. 1 (format reward = 1, answer reward = 1), Cat. 2 (format reward = 1, answer reward = 0), or Cat. 3 (format reward = 0). Results are written to `experiments/prompting_baselines/`.

Prompt	Cat. 1 (fmt+ans)	Cat. 2 (fmt only)	Cat. 3 (no fmt)	Accuracy
`question_only`	1	305	1013	0.08%
`r1_zero`	1	817	501	0.08%
`r1_zero_three_shot`	246	1007	66	18.65%

Table 1: GSM8K prompting results for OLMo-2-0425-1B ($n = 1{,}319$, temperature 1.0, seed 0).

Manual audit of Category 2. A random sample of 12–15 Cat. 2 outputs was inspected per prompt strategy.

- `question_only`: 0/12 were actually correct but misparsed. Responses either produce garbled LaTeX (e.g., phantom symbols, mismatched braces) or place the numeric answer outside a `\boxed{}`. The arithmetic itself is wrong in every sampled case.
- `r1_zero`: 0/15 were actually correct but misparsed. The model produces well-formed `</think> <answer> ..` but consistently commits arithmetic errors (e.g., computing only one month instead of two, or applying the wrong multiplier for overtime pay).
- `r1_zero_three_shot`: 0/12 were actually correct but misparsed. The model again formats correctly but reasons wrongly (e.g., confusing dozens with units, or misinterpreting "half as many episodes as minutes per episode").

Manual audit of Category 3.

- `question_only`: 0/8 correct but misparsed. The base model treats the prompt as a signal to generate additional math problems or entirely off-topic text; a `\boxed{}` answer almost never appears.
- `r1_zero`: ≈1/12 could be argued correct-but-misparsed. The dominant failure mode is the model inserting the answer *outside* the `<answer>` tag (e.g., writing the answer after `</think>` but before opening `<answer>`, or appending a "User:" turn). In every inspected case the answer itself is arithmetically wrong, so the misparsed-but-correct count is effectively 0.

- `r1_zero_three_shot`: 2/10 appear correct but misparsed. The model correctly computes the answer inside `</think>` but omits the `<answer> ... </answer>` wrapper, stopping immediately after `</think>`. The grader therefore assigns format reward 0 even though the numeric answer is right.

Example prompts and responses.

`question_only` **(Cat. 1 — the single correct response).**

Prompt: Luke is buying fabric for new curtains. There are five windows. Each window is 35 inches wide and Luke needs to buy fabric equal to 2 times the total width of the windows. How much fabric should he buy? Please put your final answer within \boxed{}.

Response: ... Total width = $35 \times 5 = 175$ inches. Total fabric = $2 \times 175 = 350$ inches. The final answer is: \boxed{350}.

`question_only` **(Cat. 3 — typical failure).**

Prompt: A pet shop had 6 cages of rodents. 3 of the cages have 10 hamsters and the other 3 have 5 guinea pigs. How many rodents did the pet shop have in total? Please put your final answer within \boxed{}.

Response: Pet Pet Pet Pet Pet Pet Pet Pet Pet

`r1_zero` **(Cat. 1).**

Prompt: ... User: James runs 12 miles a day for 5 days a week. If he runs 10 miles an hour how many hours does he run a week?
Assistant: `<think>`

Response: It would equate to 6 hours since 10 miles per hour times 6 hours per day equals 60 miles per week, and since 12 miles is the mode of the distance. `</think> <answer> 6 hours. </answer>`

`r1_zero` **(Cat. 2 — format correct, wrong answer).**

Prompt: ... User: John injured his back and needs to go to physical therapy. He went for 6 weeks, twice a week for 2 hours at a time at $125/hr. How much did the physical therapy cost?
Assistant: `<think>`

Response: John went to physical therapy for 6 weeks. Each week he went twice for 2 hours at a time. So he attended $2 \times 6 = 12$ hours total. `</think> <answer>`The cost is $125 \times 12 = \$1500$.`</answer>` [*GT: $3000; model ignored the "per session" structure.*]

`r1_zero_three_shot` **(Cat. 1).**

Prompt: ... User: The fastest car was 1.25× faster than the 2nd fastest. The 2nd fastest was 5× the average car (120 mph). How fast does the fastest car go?
Assistant: `<think>`

Response: The 2nd fastest car goes $120 \times 5 = 600$ mph. The fastest is $600 \times 1.25 = 750$ mph. `</think> <answer> 750 </answer>`

`r1_zero_three_shot` **(Cat. 2 — format correct, wrong answer).**

> **Prompt:** ... User: If Chester eats 3 eggs a day for 30 days then 5 eggs a day for 30 days, how many dozens does he need for 60 days?
> Assistant: `<think>`
>
> **Response:** $3 \times 30 + 5 \times 30 = 240$ eggs total. `</think> <answer> 240 </answer>` [*GT: 20 dozen; model forgot to divide by 12.*]

(b) Characterizing model behavior

`question_only.` The base model treats the question-only prompt as a signal to continue generating math-education content rather than to answer the posed problem. In the vast majority of outputs (76.8% Cat. 3) the model produces one of several patterns: (i) it appends additional, unrelated word problems immediately after the question; (ii) it generates LaTeX snippets, explanatory figures, or `overpic` boilerplate; or (iii) it outputs near-nonsense tokens ("Pet Pet Pet..."). The `\boxed{}` convention is essentially absent from pretraining continuations, so even when arithmetic appears it is almost never wrapped correctly. The lone Cat. 1 success happens to generate a fully formatted LaTeX solution and `\boxed{350}` because the question's phrasing happened to resemble a textbook example.

`r1_zero.` Pre-seeding `<think>` and framing the prompt as a conversation between User and Assistant substantially constrains the output: the format rate rises to 62.0%. The model mostly produces a plausible-sounding reasoning chain before `</think>`, then emits a numeric answer in `<answer>` tags. However, the arithmetic is almost always wrong: the model shortcircuits multi-step problems (e.g., computing 12 hours total instead of counting twice-weekly sessions at 2 hours each), invents irrelevant formulas, or hedges with an expression instead of a numeric value. Cat. 3 failures (38.0%) occur when the model (a) opens `<think>` but never closes it before the 512-token limit, (b) writes an answer after `</think>` but outside `<answer>` tags, or (c) appends an extra "User:" turn mid-generation, breaking the expected format.

`r1_zero_three_shot.` Three worked examples sharply focus the output distribution: the format rate rises to 95.0% and accuracy jumps to 18.65%. The demonstrations teach the model the expected trajectory length and stylistic register—short, step-by-step arithmetic chains ending with a single number inside `<answer>`—which dramatically reduces chat-style derailments. The model imitates the concise style of the demonstrations faithfully, but still commits systematic arithmetic errors (wrong multipliers, misinterpreted units, sign errors). Cat. 3 failures drop to just 5.0%; these are mostly outputs that close `</think>` but stop without ever opening `<answer>`, or that are cut off at the 512-token limit before reaching the closing `</answer>` tag. As noted above, roughly 20% of Cat. 3 cases from this prompt are actually correct answers that the parser misses because the `<answer>` wrapper is absent.

2 Group Relative Policy Optimization

2.1 Deriving on-policy GRPO

Problem (`baseline_calcs`)

Compute the variance of the policy gradient estimator (5 points).

Setup: $\pi_\theta(A=1) = p = \sigma(\theta)$, $r(A) = \mathbf{1}\{A=1\}$.

Useful identities. The sigmoid derivative gives $\nabla_\theta p = p(1-p)$. So

$$\nabla_\theta \log \pi_\theta(A=1) = \frac{p(1-p)}{p} = 1-p, \qquad \nabla_\theta \log \pi_\theta(A=0) = \frac{-p(1-p)}{1-p} = -p.$$

(a) Variance of the unadjusted estimator

Deliverable: An expression in terms of n and p, accompanied by a derivation.

Let $Z_i = r(A_i)\nabla_\theta \log \pi_\theta(A_i)$. With $r(A) = \mathbf{1}\{A=1\}$:

$$Z_i = \begin{cases} 1-p & \text{with probability } p, \\ 0 & \text{with probability } 1-p. \end{cases}$$

So $\mathbb{E}[Z_i] = p(1-p)$ and $\mathbb{E}[Z_i^2] = p(1-p)^2$. The variance of one sample is

$$\mathrm{Var}(Z_i) = p(1-p)^2 - p^2(1-p)^2 = p(1-p)^3.$$

The samples are i.i.d., so

$$\mathrm{Var}\left[\frac{1}{n}\sum_{i=1}^{n} Z_i\right] = \frac{p(1-p)^3}{n}.$$

(b) Variance of the baseline-adjusted estimator

Deliverable: An expression in terms of n, b, and p, accompanied by a derivation and some discussion.

Let $Z_i = (r(A_i) - b)\nabla_\theta \log \pi_\theta(A_i)$. Then

$$Z_i = \begin{cases} (1-b)(1-p) & \text{with probability } p, \\ b\,p & \text{with probability } 1-p. \end{cases}$$

The expectation is unchanged: $\mathbb{E}[Z_i] = p(1-b)(1-p) + (1-p)bp = p(1-p)$. The second moment is

$$\begin{aligned} \mathbb{E}[Z_i^2] &= p(1-b)^2(1-p)^2 + (1-p)\,b^2p^2 \\ &= p(1-p)\big[(1-b)^2(1-p) + b^2p\big] \\ &= p(1-p)\big[(1-p) - 2b(1-p) + b^2\big]. \end{aligned}$$

Subtract $\mathbb{E}[Z_i]^2 = p^2(1-p)^2$ and factor:

$$\begin{aligned} \mathrm{Var}(Z_i) &= p(1-p)\big[(1-p)^2 - 2b(1-p) + b^2\big] \\ &= p(1-p)\,(1-p-b)^2. \end{aligned}$$

For the mean over n samples:

$$\boxed{\mathrm{Var}\left[\frac{1}{n}\sum_{i=1}^{n}(r(A_i)-b)\nabla_\theta \log \pi_\theta(A_i)\right] = \frac{p(1-p)(1-p-b)^2}{n}.}$$

The variance is minimized at $b^\star = 1-p$, where it equals zero. The optimal baseline is therefore not the expected reward p. It is $1-p$. The reason is that the score function $\nabla_\theta \log \pi_\theta(A)$ already has nonzero magnitude on $A=0$, so the baseline must compensate for that side too.

(c) Variance with the population-mean baseline $b = p$

Substituting $b = p$ in part (b):

$$\mathrm{Var}\left[\frac{1}{n}\sum_i Z_i\right] = \frac{p(1-p)(1-2p)^2}{n}.$$

Compared to part (a), the ratio is

$$\frac{(1-2p)^2}{(1-p)^2}.$$

The baseline-adjusted variance is lower than the unadjusted variance when $p \leq 2/3$. It is higher when $p > 2/3$. At $p = 1/2$ the adjusted variance is zero. At $p \to 1$ the adjusted variance is much larger than the unadjusted one. The population-mean baseline is therefore sometimes better and sometimes worse, depending on p.

2.2 Implementing on-policy GRPO

Problem (`tokenize_prompt_and_output`)

Prompt and output tokenization (1 point).

Deliverable: Implement `tokenize_prompt_and_output`.

Test status: passing. The prompt and output are tokenized separately without special tokens. The two token lists are concatenated and right-padded to the longest sequence in the batch. The final position is dropped from `input_ids` and the first position is dropped from `labels`, giving the standard shift-by-one alignment. The `response_mask` is shifted the same way as `labels` and is true on the positions of response tokens only.

Problem (`get_response_log_probs`)

Response log-probs (and entropy) (1 point).

Deliverable: Implement `get_response_log_probs`.

Test status: passing. The model produces logits of shape (B, T, V). We take a log-softmax over the vocabulary and gather the label log-probability at each position. Entropy is computed in a numerically stable way as $-\sum_v \exp(\log p_v) \log p_v$ using the same log-softmax.

Problem (`compute_rollout_rewards`)

Computing the rewards of rollouts (1 point).

Deliverable: Implement `compute_rollout_rewards`.

Test status: passing. The reward function is called on every (response, ground truth) pair. The total, format, and answer rewards are collected. The total reward tensor is returned alongside mean reward statistics for logging.

Problem (`compute_group_normalized_rewards_grpo`)

Group normalization (1 point).

Deliverable: Implement `compute_group_normalized_rewards` with `baseline="mean"`, `advantage_normalizer="`

Test status: passing. Rewards are reshaped to (n_prompts, group_size). The per-group mean is subtracted and the per-group std is divided out. A small `advantage_eps` is added to the std before division to avoid divide-by-zero when all rewards in a group are equal.

Problem (`compute_policy_gradient_loss_on_policy`)

On-policy policy gradient (1 point).

Deliverable: Implement `compute_policy_gradient_loss` with `importance_reweighting_method = "none"`.

Test status: passing. The per-token loss is $-A \cdot \log \pi_\theta(y_t \mid x, y_{<t})$. The advantage is broadcast across the sequence dimension. Aggregation across tokens happens in `aggregate_loss_across_microbatch`.

Problem (`aggregate_loss_across_microbatch_sequence`)

Aggregate loss across tokens and sequences (0.5 points).

Deliverable: Implement `aggregate_loss_across_microbatch` with `loss_normalization = "sequence"`.

Test status: passing. The per-token loss is masked by the response mask. Each sequence is divided by its own response-token count to get a per-sequence average. The per-sequence averages are then averaged over the microbatch.

Problem (`grpo_train_step_standard_on_policy`)

GRPO train step (5 points).

Deliverable: Implement `grpo_train_step` for the standard on-policy GRPO setting.

Test status: passing. The full batch is tokenized once. Rewards and group-normalized advantages are computed once. The batch is then split into `gradient_accumulation_steps` microbatches. Each microbatch runs a forward pass and a backward pass. For sequence normalization the microbatch loss is divided by `gradient_accumulation_steps` so the accumulated gradient equals the full-batch average. For constant normalization the microbatch loss is already a partial sum and is not rescaled. The gradient norm is clipped to `max_grad_norm` before the optimizer step. Logged metrics include loss, gradient norm, mean reward, mean format reward, mean answer reward, and any per-loss metadata such as clip fraction.

2.3 Experiments

Problem (`grpo_experiments_standard_on_policy`)

Use GRPO to improve OLMo-2-0425-1B performance on GSM8K (10 points).

(a) Training script

Deliverable: A script to run GRPO (standard, on-policy) on GSM8K and OLMo-2-0425-1B.

Pointer to script: `scripts/[TODO: ...]`. Brief description of loop structure (vLLM init, wandb logging, dataset loading, rollout, gradient step, periodic validation).

(b) Sanity check (∼50 steps)

Deliverable: Evidence that convinced you the script is correct.

Insert a small plot of validation reward over the first 50 steps, and a couple of sample rollouts. Comment on whether reward is trending up.

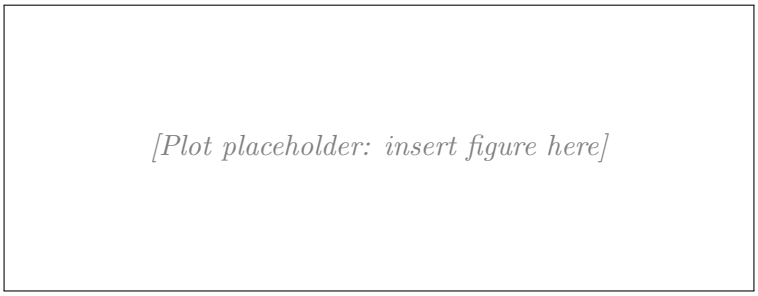
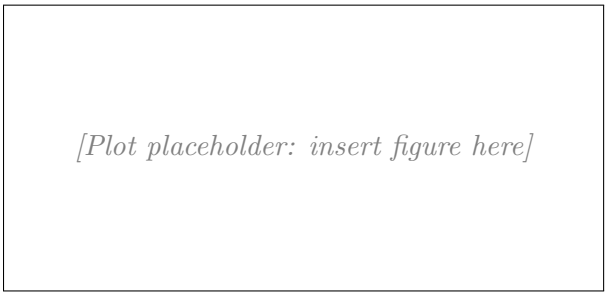


Figure 1: Validation reward, ∼50 sanity-check steps.

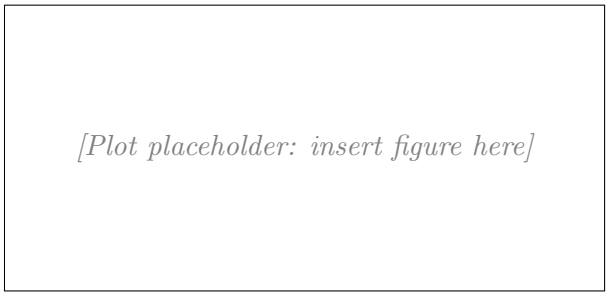
(c) Full run with 4 seeds (r1_zero, suggested hyperparameters)

Deliverable: Plots per metric, qualitative rollout examples, and a procedure that reaches $\geq 25\%$ final validation accuracy.

Required metrics: loss, gradient norm, token entropy, train rewards (total & format), val rewards (total & format), val avg response length, + optional debug metrics. Show mean±std (or min/max) across 4 seeds.

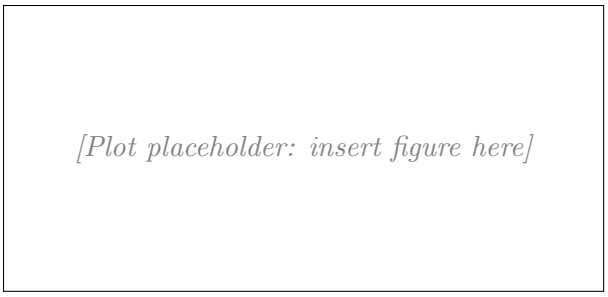


(a) Train / val total reward

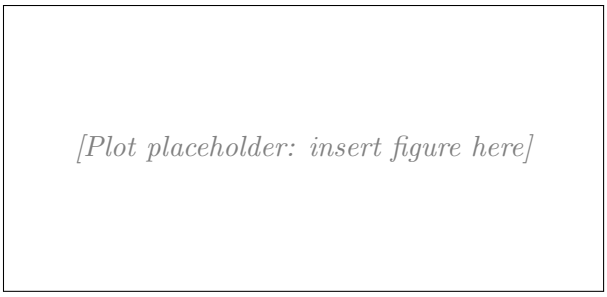


(b) Loss

Figure 2: Standard on-policy GRPO, 4 seeds.



(a) Gradient norm



(b) Token entropy

Figure 3: Diagnostics.

Rollout examples (before vs. after training).

Insert 2–3 examples each, before and after training.

Final accuracy. [TODO: report mean±std final validation accuracy.]

Problem (`grpo_learning_rate`)

Tune the learning rate (3 points).

Deliverable: Plot of final validation reward vs. learning rate, with a few sentences of commentary.

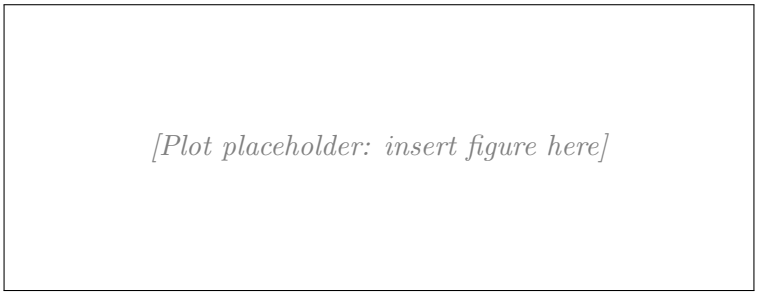


Figure 4: Final validation reward vs. learning rate.

Discuss optimum, divergence behavior, number of seeds used and why.

Problem (`grpo_prompt_ablation`)

Prompt ablation (3 points).

Deliverable: Commentary, supported by plots for referenced metrics.

Compare `question_only`, `r1_zero`, `r1_zero_three_shot`. Which has best average reward, lowest variance, systematic differences in other metrics, and how confident are you?

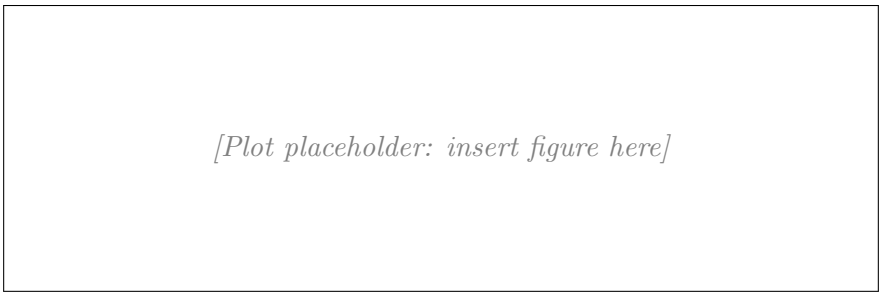


Figure 5: Prompt ablation: validation reward over training.

3 RL Algorithm Variants

3.1 Dr. GRPO

Problem (`think_about_length_normalization`)

Think about length normalization (1 point).

Deliverable: A few sentences of discussion.

Per-sequence length normalization first averages the loss over the tokens of each sequence and then averages over sequences. Each sequence contributes equally to the loss. A long sequence gets a smaller per-token weight than a short sequence.

Constant normalization divides the summed loss by a fixed number $Z = BGL$. Each token contributes equally to the loss. A long sequence contributes more to the gradient than a short sequence.

Per-sequence normalization is more robust when response lengths vary a lot inside a batch. It prevents long rollouts from dominating the gradient. The downside is a bias toward shorter sequences. Per-token weight is higher for short ones, so the model can learn to truncate.

Constant normalization is more faithful to the original policy gradient. It treats every token as an equal action and matches the derivation in Section 4.1. The downside is high variance across batches with different total token counts.

Problem (`compute_group_normalized_rewards_drgrpo`)

Dr. GRPO group normalization (0.5 points).

Deliverable: Update `compute_group_normalized_rewards` to support `advantage_normalizer = "none"` and `baseline = "none"`.

Test status: passing. The implementation branches on `baseline` and `advantage_normalizer`. The `none` options skip mean-subtraction and skip std-division respectively.

Problem (`aggregate_loss_across_microbatch_constant`)

Dr. GRPO loss aggregation (0.5 points).

Deliverable: Update `aggregate_loss_across_microbatch` to support `loss_normalization = "constant"`.

Test status: passing. The masked loss is summed over all tokens in the microbatch and divided by the fixed `normalization_constant`. The microbatch loss is therefore already a partial sum and is not rescaled by the train step.

3.2 Rejection fine tuning

Problem (`think_about_rft`)

Think about RFT (2 points).

Deliverable: A few sentences of discussion.

The RFT estimator only touches correct rollouts:

$$g_{\text{RFT}} = \frac{1}{Z} \sum_x \sum_j \mathbf{1}\{r(y^{(j)}|x) = 1\} \nabla_\theta \log \pi_\theta(y^{(j)}|x).$$

The on-policy Dr. GRPO estimator touches all rollouts but centers by the group mean μ:

$$g_{\text{Dr.GRPO}} = \frac{1}{Z} \sum_x \sum_j (r(y^{(j)}|x) - \mu) \nabla_\theta \log \pi_\theta(y^{(j)}|x).$$

Expectation. Both have the same expectation in the on-policy limit. For binary reward,

$$\mathbb{E}_{y\sim\pi_\theta}[\mathbf{1}\{r = 1\}\nabla \log \pi_\theta(y|x)] = \mathbb{E}_y[r\nabla \log \pi_\theta(y|x)] = \nabla_\theta \eta(x),$$

where $\eta(x) = \mathbb{E}_y[r(y|x)]$. The mean-subtracted Dr. GRPO term also reduces to $\nabla\eta(x)$ in the $G \to \infty$ limit because $\mu \to \eta(x)$ and $\mathbb{E}_y[\nabla \log \pi_\theta] = 0$.

Variance. Dr. GRPO subtracts the group mean. This is a valid baseline. It reduces variance when the reward and the score function are positively correlated, which is the common case. RFT keeps a positive log-prob term on every correct sample and zero on every incorrect one. Its variance is generally higher because it skips the baseline correction.

When to prefer each. RFT is simpler and more stable when the group reward is near 0 or 1, since μ goes to a constant and Dr. GRPO's centering does almost nothing. RFT is also a natural choice if you only have access to correct rollouts. Dr. GRPO is preferable when groups have mixed rewards, since the baseline gives a real variance reduction.

3.3 MaxRL

Problem (`derive_difficulty_reweightings`)

Derive difficulty reweightings induced by different advantage normalizers (6 points).

Setup. Write $\eta(x) = \mathbb{E}_{y\sim\pi_\theta}[r(y|x)]$. The standard policy gradient is $\nabla_\theta J_\theta = \mathbb{E}_x[\nabla\eta(x)]$. The surrogate target is $\nabla_\theta J_{\theta,w} = \mathbb{E}_x[w(x)\nabla\eta(x)]$. The inner expectation over $\sim \pi_\theta(\cdot|x)$ has the identity $\mathbb{E}_y[\nabla \log \pi_\theta(y|x)] = 0$. So for any function $f(x)$ that does not depend on y, $\mathbb{E}_y[f(x)\nabla \log \pi_\theta(y|x)] = 0$.

(a) Dr. GRPO reweighting

Deliverable: An expression in terms of a subset of the problem parameters, with a few sentences of justification.

With $Z = G$ and $G \to \infty$:

$$\frac{1}{G}\sum_{j=1}^{G}(r(y^{(j)}|x) - \mu)\nabla_\theta \log \pi_\theta(y^{(j)}|x) \;\longrightarrow\; \mathbb{E}_y\big[(r - \eta(x))\nabla \log \pi_\theta(y|x)\big].$$

The mean-subtracted term gives zero contribution since $\eta(x)$ does not depend on y. So the limit equals $\mathbb{E}_y[r\nabla \log \pi_\theta(y|x)] = \nabla\eta(x)$. Taking expectation over x:

$$\boxed{w(x) = 1.}$$

Dr. GRPO weights all prompts equally. It matches the unbiased policy gradient in the on-policy limit.

(b) GRPO (with std normalization) reweighting

Deliverable: An expression with derivation.

The estimator divides by the group std. For binary reward,

$$\text{std} \xrightarrow{G\to\infty} \sqrt{\eta(x)(1-\eta(x))}.$$

The numerator converges to $\nabla\eta(x)$ as in (a). Combining:

$$\mathbb{E}_x\Big[\frac{\nabla\eta(x)}{\sqrt{\eta(x)(1-\eta(x))}}\Big] = \mathbb{E}_x[w(x)\nabla\eta(x)] \;\Rightarrow\; \boxed{w(x) = \frac{1}{\sqrt{\eta(x)(1-\eta(x))}}.}$$

The weight is large at $\eta \approx 0$ and $\eta \approx 1$ and small at $\eta = 1/2$. GRPO upweights very easy and very hard prompts.

(c) MaxRL reweighting (divide by group mean)

Deliverable: An expression with derivation.

The estimator divides by μ. With $G \to \infty$, $\mu \to \eta(x)$. So

$$\mathbb{E}_x\Big[\frac{\nabla\eta(x)}{\eta(x)}\Big] = \mathbb{E}_x[w(x)\nabla\eta(x)] \;\Rightarrow\; \boxed{w(x) = \frac{1}{\eta(x)}.}$$

The weight is large for hard prompts (small η) and small for easy prompts. MaxRL upweights hard prompts. Note that the surrogate is $\mathbb{E}_x[\nabla \log \eta(x)]$, i.e. the gradient of the log expected reward.

Problem (`think_about_advantage_normalization`)

Think about advantage normalization (2 points).

Deliverable: A few sentences of discussion.

The three options give different difficulty reweightings (see the derivation above). They also have different bias and variance.

std normalization keeps the gradient norm roughly constant across groups. Every group contributes a similar update size. This is good for stability. The cost is a bias toward easy and hard prompts. The std blows up the weight at the extremes of η.

mean normalization (MaxRL) upweights hard prompts. This is good for curriculum-like training, since hard prompts are exactly the ones the model needs to learn. The cost is instability when μ is small. The weight $1/\mu$ can grow without bound.

no normalization (Dr. GRPO) is the unbiased policy gradient and weights all prompts equally. This is the safest choice in theory. The cost is high variance in gradient norm across groups.

Use std when training is unstable and prompt difficulty is roughly uniform. Use mean when the dataset has many easy prompts and you want the model to focus on the hard tail. Use none when you want unbiased estimation and have enough samples to absorb the variance.

Problem (`compute_group_normalized_rewards_maxrl`)

MaxRL group normalization (0.5 points).

Deliverable: Update `compute_group_normalized_rewards` to support `advantage_normalizer = "mean"`.

Test status: passing. The branch divides the centered advantage by the per-group mean plus `advantage_eps`.

3.4 Experiments

Problem (`grpo_train_step_variants_on_policy`)

GRPO train step variants (2.5 points).

Deliverable: Update `grpo_train_step` to support the full range of on-policy variants.

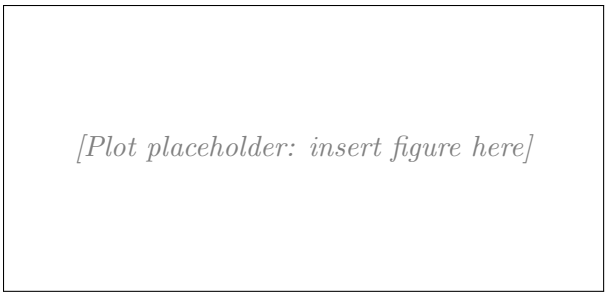
Test status: passing for all four variants (`grpo_constant`, `dr_grpo`, `rft`, `maxrl`). The train step takes `baseline`, `advantage_normalizer`, and `loss_normalization` as arguments and dispatches to the right code path. Zero-advantage sequences can be skipped by checking `advantages != 0` after group normalization. This saves a forward and a backward pass for groups with all-equal rewards (under `baseline="mean"`) and for incorrect samples (under `baseline="none"`).

Problem (`grpo_experiments_variants_on_policy`)

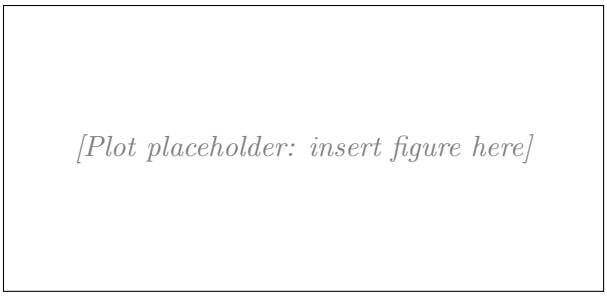
Compare the performance of different RL algorithms (10 points).

Deliverable: Commentary, supported by plots for referenced metrics.

Variants: `GRPO_constant`, `Dr_GRPO`, `RFT`, `MaxRL` (with constant normalization). 4 seeds each, zero-shot `r1_zero`. Compare against the standard GRPO runs.



(a) Validation reward across variants



(b) Token entropy across variants

Figure 6: On-policy variant comparison (4 seeds).

Discussion.

Which method has best average performance? Lowest variance? Which would benefit from more hyperparameter tuning? How confident are you in the findings given between-seed variance?

4 Off-policy RL

4.1 Importance reweighting / clipping

Problem (`derive_surrogate_objectives`)

Derive surrogate objectives for importance reweighting approaches (2 points).

Deliverable: An expression in terms of a subset of the problem parameters, accompanied by a derivation.

Setup. The pairwise estimator groups tokens into pairs (y_{2t-1}, y_{2t}). For each pair index $t \in \{1, \dots, L/2\}$ define a mixed policy. It samples all pairs from π_{θ_0} except pair t, which is sampled from π_θ:

$$\tilde{\pi}_t(y|x) = \Big(\prod_{s \neq t} \pi_{\theta_0}(y_{2s-1}|x, y_{<2s-1})\pi_{\theta_0}(y_{2s}|x, y_{<2s})\Big) \cdot \pi_\theta(y_{2t-1}|x, y_{<2t-1})\pi_\theta(y_{2t}|x, y_{<2t}).$$

Surrogate. The surrogate objective sums the reward under each mixed policy:

$$\boxed{J_\theta^{\text{pair}} = \mathbb{E}_x \left[\sum_{t=1}^{L/2} \mathbb{E}_{y \sim \tilde{\pi}_t}\big[r(y|x)\big] \right].}$$

Derivation. Take the gradient and reweight each term so it is sampled from π_{θ_0} everywhere. For a fixed pair t, the ratio of $\tilde{\pi}_t$ to π_{θ_0} is exactly the pair t ratio:

$$\frac{\tilde{\pi}_t(y|x)}{\pi_{\theta_0}(y|x)} = \frac{\pi_\theta(y_{2t-1}|x, y_{<2t-1})\pi_\theta(y_{2t}|x, y_{<2t})}{\pi_{\theta_0}(y_{2t-1}|x, y_{<2t-1})\pi_{\theta_0}(y_{2t}|x, y_{<2t})}.$$

Applying the log-derivative trick on the pair t factors only (the other factors do not depend on θ):

$$\nabla_\theta \mathbb{E}_{y\sim\tilde{\pi}_t}[r(y|x)] = \mathbb{E}_{y\sim\pi_{\theta_0}}\Big[\tfrac{\tilde{\pi}_t(y|x)}{\pi_{\theta_0}(y|x)}\, r(y|x)\, \nabla_\theta \log\big(\pi_\theta(y_{2t-1}|x, y_{<2t-1})\pi_\theta(y_{2t}|x, y_{<2t})\big)\Big].$$

Summing over t and over x recovers the given pairwise estimator. So the surrogate above is the objective whose gradient equals that estimator.

4.2 Implementation

Problem (`compute_policy_gradient_loss_off_policy`)

Off-policy policy gradient with token-level reweighting (1 point).

Deliverable: Update `compute_policy_gradient_loss` to support `importance_reweighting_method = "noclip"` or `"grpo"`.

Test status: passing for both `noclip` and `grpo`. The per-token importance ratio is $\exp(\log \pi_\theta(y_t) - \log \pi_{\theta_0}(y_t))$. `noclip` multiplies the advantage by this ratio. `grpo` also computes a clipped ratio and takes the per-token minimum of $A \cdot \rho$ and $A \cdot \text{clip}(\rho, 1-\varepsilon, 1+\varepsilon)$. The clip fraction is logged for diagnostics.

Problem (`compute_policy_gradient_loss_off_policy_gspo`)

Off-policy policy gradient with sequence-level reweighting (1 point).

Deliverable: Update `compute_policy_gradient_loss` to support `importance_reweighting_method = "gspo"`.

Test status: passing. The geometric mean of the per-token ratio is computed in log space. The per-token log-ratio is averaged over the response tokens (masked sum divided by response-token count), then exponentiated. This avoids overflow from multiplying many small ratios. The clipped sequence ratio is used the same way as in the token-level case.

Problem (`grpo_train_step_off_policy`)

Off-policy GRPO train step (2.5 points).

Deliverable: Update `grpo_train_step` to support off-policy arguments.

Test status: passing for `noclip`, `grpo`, and `gspo`. The train step accepts `old_log_probs`, slices them per microbatch, and forwards them to the loss function. The rest of the training loop is unchanged.

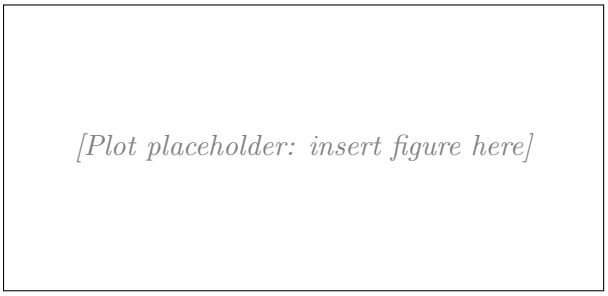
4.3 Experiments

Problem (`grpo_experiments_off_policy`)

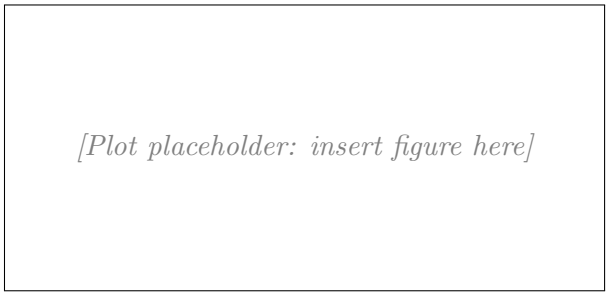
Compare the performance of different off-policy algorithms (10 points).

Deliverable: Commentary, supported by plots for referenced metrics.

Variants: `offpolicy_naive`, `offpolicy_noclip`, `offpolicy_clip` ($\varepsilon = 0.2$), `offpolicy_gspo` ($\varepsilon = 3\times10^{-4}$). 4 seeds, zero-shot `r1_zero`. Include clip fraction in logged metrics.



(a) Validation reward across off-policy variants



(b) Clip fraction

Figure 7: Off-policy variant comparison (4 seeds).

Discussion.

Compared to on-policy GRPO, how do these methods perform? Effect on stability and variance? Which clipping method seems more stable? Methods that might benefit from more tuning? Confidence in findings.

Problem (`think_about_importance_reweighting`)

Think about importance reweighting (2 points).

Deliverable: A few sentences of commentary.

(a) No reweighting. Highest bias, lowest variance. The estimator ignores the gap between π_θ and π_{θ_0}. It works well when π_θ and π_{θ_0} are close, for example after only a few off-policy steps. It breaks down once the policies drift apart.

(b) PPO/GRPO token-level clipped. Medium bias, medium variance. Token-level ratios are exponential in length and would normally explode. Clipping caps each ratio in $[1-\varepsilon, 1+\varepsilon]$, which controls variance. The clip cost is bias: gradients on tokens with extreme ratios are zeroed out. This is the standard choice for off-policy LLM RL.

(c) GSPO sequence-level clipped. Lower bias on the sequence-level objective than (b), but the raw sequence ratio has very high variance. GSPO uses the geometric mean over tokens and clips with a tiny ε. This is appropriate when sequences are long and the credit should be at the sequence level, not the token level.

In short: pick (a) for nearly on-policy training, (b) for the standard off-policy LLM setting, and (c) for long sequences with sequence-level credit assignment.

5 Try your own policy gradient estimator

Problem (`try_your_own`)

Try your own policy gradient estimator (10 points).

Deliverable: Commentary, plots, and optionally mathematical derivations.

Proposed estimator

Describe your change relative to one of the approaches above (advantage estimator / importance reweighting / baseline). Change only one thing relative to the chosen baseline.

$$\hat{g} = \textbf{[TODO: your estimator]}.$$

Theoretical motivation

Intuition or derivation that motivates the change.

Results

Compare against baselines on OLMo-2-0425-1B / GSM8K with multiple random seeds and shared hyperparameters.

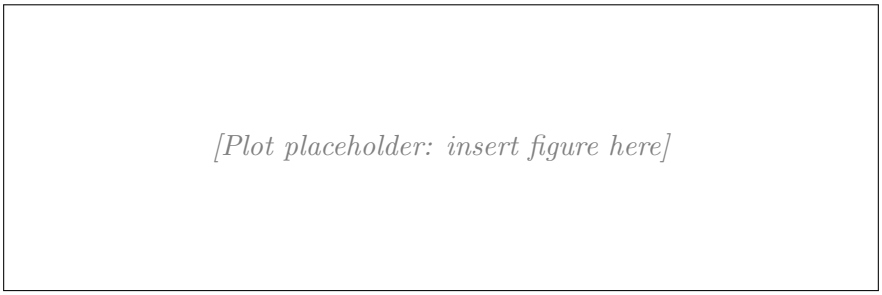


Figure 8: Custom estimator vs. baselines (validation reward).

Does it beat the baselines? [TODO: yes/no, with discussion.]